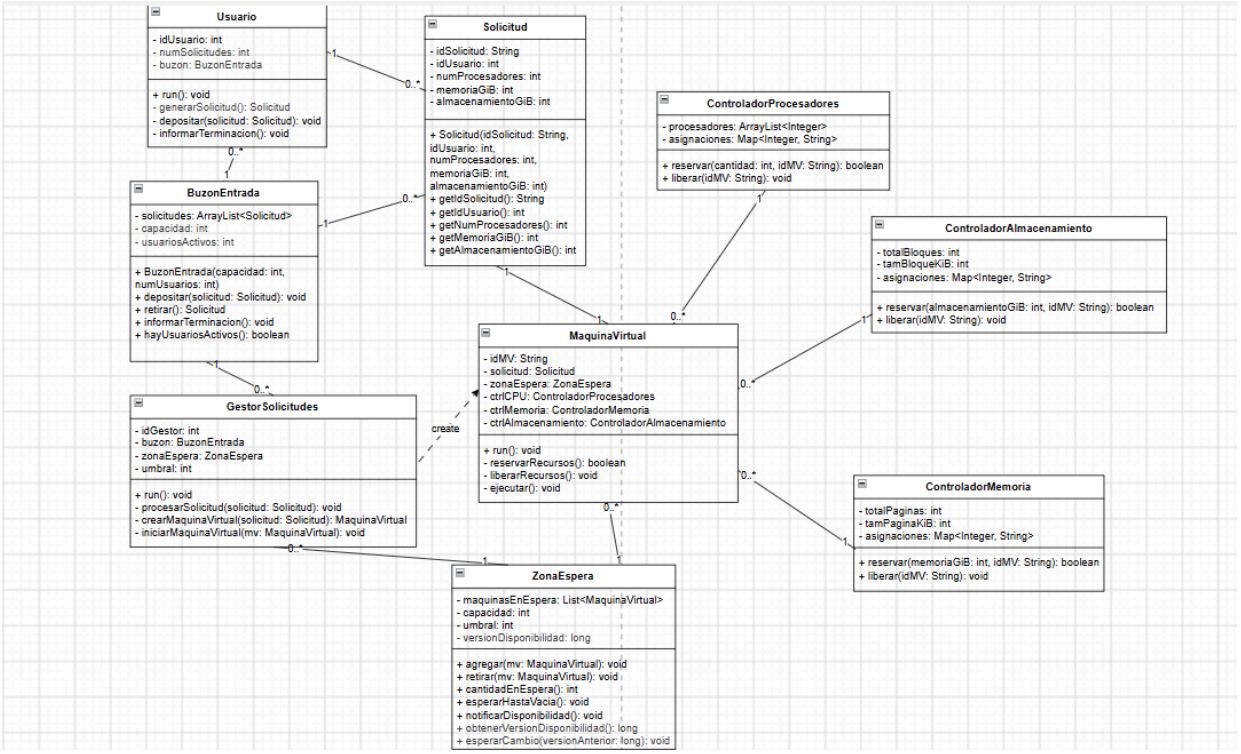
Universidad de los Andes

Documento Caso 3 TIC – ISIS 1311

Santiago Tarazona Upegui 202513771

Sarita Martinez Pedrozo 202510428

Diagrama de clases:



Para una mejor visualización, el archivo draw.io está adjunto en la implementación del proyecto.

Explicación del funcionamiento del proyecto:

1. **Usuario:** La clase *Usuario* representa a cada uno de los productores de solicitudes del sistema. Sus atributos permiten identificar al usuario, determinar cuántas solicitudes debe generar y mantener una referencia al buzón de entrada en el que las depositará. El método run() coordina su ejecución como un hilo independiente, mientras que generarSolicitud() construye cada solicitud y depositar() se encarga de incorporarla al

buzón. Finalmente, informarTerminacion() comunica que el usuario ha terminado de producir solicitudes, permitiendo que el sistema actualice el registro de usuarios activos. Esta organización separa la generación de solicitudes de su almacenamiento y de la coordinación entre hilos, responsabilidades que corresponden a otras clases. De esta manera, *Usuario* se concentra simplemente en producir solicitudes y enviarlas al sistema de forma coordinada.

- **Solicitud:** La clase *Solicitud* representa cada petición de creación de una máquina virtual y almacena la información necesaria para identificarla y conocer los recursos que requiere. Sus atributos incluyen el identificador de la solicitud, el identificador del usuario que la genera, la cantidad de procesadores solicitados y las cantidades de memoria y almacenamiento expresadas en GiB. Los métodos de consulta permiten que los gestores y las máquinas virtuales accedan a esta información sin modificar directamente los atributos del objeto. Se propone mantener la solicitud únicamente como una representación de datos, sin asignarle responsabilidades relacionadas con la ejecución de hilos, la administración de recursos o la sincronización. Su generación corresponde a *Usuario*, mientras que su almacenamiento temporal es responsabilidad de *BuzonEntrada*. Esta separación permite que cada solicitud conserve la información necesaria durante su procesamiento y facilita la comunicación entre los componentes del sistema ya mencionados.
- **BuzonEntrada:** La clase *BuzonEntrada* administra el almacenamiento temporal de las solicitudes generadas por los usuarios y pendientes de procesamiento por los gestores. Sus atributos incluyen una colección de solicitudes, su capacidad máxima y un contador de usuarios activos. La colección se implementará mediante un ArrayList, manteniendo el orden de llegada para procesar las solicitudes siguiendo una política FIFO, es decir que, la primera solicitud depositada será la primera en retirarse. El método depositar() incorpora nuevas solicitudes, mientras que retirar() permite a los gestores obtenerlas para su procesamiento. Estas operaciones deben coordinarse mediante sincronización para evitar modificaciones simultáneas (condiciones de carrera) y permitir que los productores esperen cuando el buzón esté lleno y los consumidores cuando esté vacío, pero todavía puedan llegar solicitudes. Por su parte, informarTerminacion() actualiza el número de usuarios activos y notifica a los hilos que puedan estar esperando. De esta manera, los gestores pueden

determinar cuándo no quedan solicitudes por procesar ni usuarios que puedan generar nuevas, garantizando una finalización correcta del sistema.

- **GestorSolicitudes:** La clase *GestorSolicitudes* representa a cada uno de los gestores encargados de procesar las solicitudes depositadas por los usuarios. Sus atributos permiten identificar al gestor y acceder tanto al buzón de entrada como a la zona de espera. El método run() coordina su ejecución como hilo independiente y organiza el procesamiento de las solicitudes disponibles. Para mantener separadas las responsabilidades, procesarSolicitud() gestiona cada petición, crearMaquinaVirtual() construye la máquina virtual correspondiente e iniciarMaquinaVirtual() inicia su ejecución. Antes de continuar procesando solicitudes, el gestor consulta la cantidad de máquinas virtuales pendientes en la zona de espera. Si esta cantidad alcanza o supera el umbral, debe esperar hasta que todas las máquinas virtuales pendientes correspondientes hayan sido procesadas, de acuerdo con la condición establecida en el enunciado. El gestor finaliza cuando todos los usuarios han terminado y el buzón está vacío. Así, la clase coordina el consumo de solicitudes y la creación de máquinas virtuales sin asumir directamente la administración de los recursos físicos.
- **MaquinaVirtual:** La clase *MaquinaVirtual* representa cada MV creada a partir de una solicitud y se ejecuta como un hilo independiente. Sus atributos permiten identificarla, consultar la solicitud que la originó y acceder a la zona de espera y a los controladores de procesadores, memoria y almacenamiento. El método run() coordina su ciclo de vida, mientras que reservarRecursos() intenta obtener los recursos requeridos en el orden establecido: procesadores, memoria y almacenamiento. Antes de intentar la reserva, consulta la versión de disponibilidad de la zona de espera para identificar posibles cambios en los recursos. Si alguna reserva falla, la máquina virtual debe liberar todos los recursos obtenidos previamente para evitar mantener asignaciones que impidan el uso de los recursos para otras MV, y posteriormente ingresar a la zona de espera para volver a intentarlo. Para ello, utiliza la versión registrada y comprueba si hubo algún cambio antes de quedar esperando, evitando perder notificaciones que puedan ocurrir antes de comenzar la espera. Una vez consigue todos los recursos, se ejecuta y, al finalizar, los libera mediante liberarRecursos(), notificando a las máquinas virtuales que se encuentran esperando. El método ejecutar() representa el periodo de actividad de la máquina virtual. Esta organización concentra en una sola clase la coordinación del ciclo de vida de cada

MV, mientras que la administración y sincronización de los recursos compartidos queda a cargo de sus respectivos controladores.

- **ZonaEspera:** La clase *ZonaEspera* administra el registro de las máquinas virtuales que no han conseguido reservar todos los recursos necesarios para comenzar su ejecución. Sus atributos incluyen una colección de máquinas virtuales pendientes, la capacidad máxima de la zona, el umbral utilizado por los gestores y una variable de versión que permite registrar los cambios en la disponibilidad de los recursos como un contador. El método agregar() incorpora las máquinas virtuales que necesitan esperar, evitando registros duplicados y respetando la capacidad máxima de la zona, mientras que retirar() elimina aquellas que consiguen completar su reserva. Por su parte, cantidadEnEspera() permite consultar cuántas máquinas virtuales permanecen pendientes y esperarPermisoProcesamiento() permite que los gestores comprueben si pueden continuar procesando solicitudes. Cuando se alcanza el umbral, los gestores esperan pasivamente hasta que no queden máquinas virtuales pendientes ni creaciones en curso. El método notificarDisponibilidad() incrementa la versión de disponibilidad y despierta a los hilos que esperan sobre la zona. Finalmente, obtenerVersionDisponibilidad() permite consultar la versión actual y esperarCambio() permite que una máquina virtual espere hasta que se registre un cambio, evitando perder notificaciones que puedan ocurrir antes de comenzar la espera. Las operaciones sobre la colección y las condiciones de espera deben estar sincronizadas para evitar inconsistencias y coordinar correctamente los hilos.
- **ControladorProcesadores**: La clase *ControladorProcesadores* administra los procesadores disponibles en el sistema y registra cuáles están asignados a cada máquina virtual. Para ello, el atributo procesadores almacena los identificadores de los procesadores físicos, mientras que asignaciones relaciona cada procesador ocupado con el identificador de la MV que lo utiliza. Se emplea una lista para representar el conjunto de procesadores y un mapa para registrar sus asignaciones, considerando libres aquellos que no aparecen en dicho registro. El método reservar() comprueba si existe una cantidad suficiente de procesadores disponibles y, si es posible completar la solicitud, asigna los procesadores correspondientes a la MV, en caso de que no hayan, no realiza ninguna asignación y devuelve false. Por su parte, liberar() elimina las asignaciones asociadas a una MV y devuelve esos procesadores a la disponibilidad del sistema. Ambas operaciones deben estar sincronizadas para que

la comprobación de disponibilidad y la asignación se realicen de manera coordinada, evitando que varias máquinas virtuales reserven simultáneamente los mismos procesadores (condición de carrera). Así, el controlador garantiza la consistencia del registro y centraliza la administración de este recurso compartido.

- **ControladorMemoria**: La clase *ControladorMemoria* administra la memoria física disponible y registra las páginas asignadas a cada máquina virtual. El atributo totalPaginas representa la cantidad total de páginas que puede gestionar, mientras que tamPaginaKiB almacena el tamaño de cada página y permite convertir la memoria solicitada, expresada en GiB, a la cantidad de páginas necesarias. El atributo asignaciones utiliza un mapa para relacionar cada página ocupada con el identificador de la MV que la utiliza, las páginas que no aparecen en este registro se consideran libres. El método reservar() calcula las páginas requeridas, comprueba su disponibilidad y, si existen suficientes, las asigna a la máquina virtual solicitante. Si no puede completar la reserva, no debe realizar asignaciones parciales y devuelve false. Por su parte, liberar() elimina las asignaciones correspondientes a una MV y devuelve esas páginas a la disponibilidad del sistema. Ambas operaciones deben estar sincronizadas para mantener la consistencia del registro y evitar que varias máquinas virtuales asignen simultáneamente las mismas páginas.
- **ControladorAlmacenamiento:** La clase *ControladorAlmacenamiento* administra el espacio de almacenamiento disponible y registra los bloques asignados a cada máquina virtual. El atributo totalBloques representa la cantidad total de bloques que puede gestionar, mientras que tamBloqueKiB almacena el tamaño de cada bloque y permite convertir las solicitudes expresadas en GiB a la cantidad de bloques necesarios. El atributo asignaciones utiliza un mapa para relacionar cada bloque ocupado con el identificador de la MV que lo utiliza, considerando libres los bloques que no aparecen en este registro. El método reservar() calcula los bloques requeridos, comprueba su disponibilidad y, si existen suficientes, los asigna a la máquina virtual solicitante. Si no puede completar la reserva, no debe efectuar asignaciones parciales y devuelve false. Por su parte, liberar() elimina las asignaciones correspondientes a una MV y devuelve esos bloques a la disponibilidad del sistema. Ambas operaciones deben estar sincronizadas para evitar asignaciones duplicadas y mantener la consistencia del registro cuando varias máquinas virtuales solicitan o liberan almacenamiento simultáneamente.

Cabe recalcar que la clase “Main” no se incluye en el diagrama de clases debido a que su responsabilidad principal consiste en inicializar y poner en marcha el sistema. Desde esta clase se configuran los parámetros de ejecución, se crean las instancias compartidas necesarias, se construyen los usuarios y gestores y se inicia la ejecución de los diferentes hilos. Por tanto, su función corresponde principalmente a la preparación y coordinación inicial del programa, mientras que el diagrama se concentra en representar los componentes que participan directamente en el funcionamiento del sistema, sus responsabilidades y las relaciones entre ellos, permitiendo mantener el diagrama enfocado en la arquitectura principal del sistema.

Relaciones entre clases

El diagrama de clases establece las relaciones necesarias para representar la interacción entre los componentes del sistema. Estas relaciones permiten identificar cómo se comunican los usuarios con el buzón de entrada, cómo los gestores procesan las solicitudes, cómo se crean las máquinas virtuales y cómo estas acceden a los recursos compartidos. A continuación, se describen las relaciones establecidas, sus multiplicidades y la justificación.

1. Usuario — BuzonEntrada: Esta relación representa la comunicación entre los usuarios y el buzón de entrada, mediante el cual depositan las solicitudes que generan. La multiplicidad establece que cada usuario utiliza un único buzón de entrada, mientras que un mismo buzón puede ser utilizado por varios usuarios para depositar sus solicitudes. Esta relación permite representar el uso de un buffer compartido por todos los productores del sistema, y su propósito es centralizar el almacenamiento temporal de las solicitudes y permitir que los gestores las retiren posteriormente.

2. Usuario — Solicitud: Esta relación representa el vínculo entre cada usuario y las solicitudes que genera. Se establece que cada solicitud corresponde a un único usuario, mientras que un usuario puede generar varias solicitudes durante su ejecución. Esta asociación permite identificar el origen de cada petición y mantener la relación entre el productor y la información que genera.

3. BuzonEntrada — Solicitud: Esta relación representa el almacenamiento de las solicitudes dentro del buzón de entrada. Un buzón puede contener cero o muchas solicitudes, mientras que cada solicitud está asociada a un único buzón. Esta relación refleja el funcionamiento del buzón como un buffer compartido y limitado, en el que los usuarios depositan sus peticiones y los gestores las retiran para iniciar su procesamiento.

4. BuzonEntrada — GestorSolicitudes: Esta relación representa la interacción entre el buzón de entrada y los gestores encargados de procesar las solicitudes. Cada gestor mantiene una referencia al buzón del cual obtiene las peticiones pendientes, mientras que un mismo buzón puede ser utilizado por varios gestores. Esto permite que los gestores trabajen concurrentemente sobre un buffer compartido, retirando solicitudes de manera coordinada y evitando que una misma petición sea procesada más de una vez, garantizando la consistencia de la información compartida.

5. GestorSolicitudes — ZonaEspera: Esta relación representa la interacción de los gestores con la zona de espera. Cada gestor mantiene una referencia a una única zona de espera, mientras que esta puede ser consultada por varios gestores. A través de esta relación, los gestores pueden conocer la cantidad de máquinas virtuales que se encuentran pendientes de obtener los recursos necesarios y compararla con el umbral establecido. De acuerdo con esta condición, determinan si pueden continuar procesando solicitudes del buzón o si deben aplicar la política de espera total.

6. GestorSolicitudes — MaquinaVirtual: Esta relación representa la creación de máquinas virtuales por parte de los gestores. Cuando un gestor procesa una solicitud, crea la máquina virtual correspondiente con la configuración de recursos indicada e inicia su ejecución. En el diagrama se representa mediante una dependencia con la palabra “create”, que indica que el gestor utiliza la clase de máquina virtual para crear nuevos objetos de ese tipo. Esta relación refleja la responsabilidad del gestor de transformar las solicitudes procesadas en máquinas virtuales, sin encargarse directamente de administrar los recursos físicos que estas necesitan.

7. Solicitud — MaquinaVirtual: Esta relación representa el vínculo entre una solicitud y la máquina virtual creada a partir de ella. Se establece una relación uno a uno, ya que cada máquina virtual se crea utilizando la configuración de una única solicitud y cada solicitud procesada da origen a una máquina virtual. La MV mantiene una referencia a la solicitud que

la originó, lo que permite consultar los recursos requeridos durante su configuración y ejecución.

8. ZonaEspera — MaquinaVirtual: Esta relación representa el registro de las máquinas virtuales que no han conseguido reservar todos los recursos necesarios para comenzar su ejecución. Cada máquina virtual utiliza una única zona de espera, mientras que una zona puede contener varias máquinas virtuales pendientes simultáneamente. Esta relación permite que la zona mantenga las referencias de las MVs que se encuentran esperando y que los gestores puedan consultar cuántas permanecen pendientes. Cuando cambia la disponibilidad de los recursos, las máquinas virtuales pueden volver a intentar completar su configuración y dejar de estar registradas en la zona una vez consiguen reservar los recursos necesarios.

9. MaquinaVirtual — ControladorProcesadores: Esta relación representa el acceso de las máquinas virtuales al controlador encargado de administrar los procesadores físicos del sistema. Cada máquina virtual utiliza una única instancia del controlador de procesadores, mientras que un mismo controlador puede ser utilizado por múltiples máquinas virtuales. Esto permite centralizar el registro de los procesadores disponibles y asignados, evitando que cada MV administre una copia independiente de los recursos. Las operaciones de reserva y liberación se realizan mediante el controlador, que debe sincronizar el acceso a la información compartida para evitar asignaciones duplicadas.

10. MaquinaVirtual — ControladorMemoria: Esta relación representa el acceso de las máquinas virtuales al controlador encargado de administrar la memoria física del sistema. Cada máquina virtual utiliza una única instancia del controlador de memoria, mientras que esta puede ser compartida por múltiples máquinas virtuales. El controlador mantiene el registro de las páginas asignadas y disponibles, además de realizar la conversión de las solicitudes expresadas en GiB a la cantidad correspondiente de páginas. De esta manera, todas las máquinas virtuales consultan y modifican un único registro centralizado.

11. MaquinaVirtual — ControladorAlmacenamiento: Esta relación representa el acceso de las máquinas virtuales al controlador encargado de administrar el espacio de almacenamiento del sistema. Cada máquina virtual utiliza una única instancia del controlador de almacenamiento, mientras que un mismo controlador puede atender las solicitudes de varias máquinas virtuales. Este objeto mantiene el registro de los bloques disponibles y

asignados, identificando qué máquina virtual utiliza cada bloque y realizando la conversión de las solicitudes expresadas en GiB a la cantidad correspondiente de bloques.

Consideraciones generales de concurrencia

El sistema utiliza hilos independientes para representar a los usuarios, los gestores y las máquinas virtuales. Como varios de estos hilos interactúan con objetos compartidos, las operaciones que modifican o consultan estados relevantes deben coordinarse mediante mecanismos de sincronización adecuados. En particular, el buzón debe garantizar la correcta inserción y retirada de solicitudes, mientras que los controladores deben proteger las operaciones de reserva y liberación de recursos. La zona de espera, por su parte, debe coordinar el registro de las máquinas virtuales pendientes, las notificaciones de disponibilidad y la condición de espera total de los gestores. En el caso de los usuarios y las máquinas virtuales, estos realizan espera pasiva cuando no pueden continuar, mientras los gestores utilizan espera semi-activa cuando el buzón está vacío. Cuando se alcanza el umbral de máquinas pendientes, los gestores pasan a una espera pasiva hasta que se resuelvan. Finalmente, cada máquina virtual debe reservar todos los recursos requeridos o liberar aquellos que haya obtenido previamente. Estas decisiones buscan mantener la consistencia de los datos compartidos, evitar asignaciones duplicadas y permitir que los hilos progresen sin introducir esperas innecesarias.

Preguntas de Análisis

1. Si el número de MVs en espera es igual o superior al umbral, el gestor debe esperar a que todas las solicitudes pendientes se procesen (no permite el ingreso de solicitudes adicionales hasta terminar con las pendientes). ¿Qué problema resuelve esta manera de procesar solicitudes?

R/ Esta medida evita que se sigan creando máquinas virtuales cuando en ese momento ya hay varias esperando recursos. De esta manera, el sistema puede concentrarse en atender las que están pendientes antes de recibir nuevas solicitudes, evitando que el sistema se sobrecargue y que aumente la competencia por los recursos disponibles

2. Una máquina reserva los recursos que necesita -uno a uno- y siempre en el orden establecido: procesador, memoria y disco. ¿Por qué es importante que todas sigan el mismo orden de solicitud?

R/ Porque permite que todas las máquinas virtuales sigan el mismo procedimiento al solicitar procesadores, memoria y almacenamiento, esto ayuda a evitar ciertos bloqueos, que pueden ocurrir cuando varias máquinas tienen recursos reservados y quedan esperando otros que están siendo utilizados, además, ayuda a la organización y el control de los recursos.

3. Si, después de haber recibido uno o más recursos, encuentra que el siguiente recurso que necesita no está disponible, debe liberar inmediatamente todos los recursos que había reservado previamente. Por ejemplo, si una máquina ya recibió procesadores y memoria, pero no puede obtener el disco requerido, deberá devolver tanto los procesadores como la memoria. ¿Por qué es importante liberar los recursos asignados cuando no se logra completar la reserva?

R/ Porque si una máquina virtual no consigue todos los recursos que necesita, no puede comenzar a ejecutarse, por lo que mantener los recursos que ya reservó solo impediría que otras máquinas los pudieran utilizar. Por eso, es importante liberarlos para que vuelvan a estar disponibles y otras máquinas puedan continuar con su propia ejecución.

4. Cuando termine su ejecución, la máquina deberá liberar todos los recursos que tenía asignados y despertar otra máquina para que use los recursos. ¿debería despertar una sola o todas? ¿por qué?

R/ Es mejor despertar a todas las máquinas virtuales que están esperando, ya que no todas necesitan la misma cantidad de recursos, por lo que si se despierta solamente una, puede que todavía no tenga los suficientes para continuar, mientras que otra sí podría hacerlo. Al despertarlas a todas, cada una puede revisar nuevamente si hay recursos disponibles y si todavía no consigue los necesarios, vuelve a esperar

Universidad de los Andes

Sincronización entre objetos

Usuario y Solicitud: Cada usuario genera sus propias solicitudes, por lo que no es necesario utilizar sincronización entre estas clases, cada solicitud contiene los datos necesarios para crear una máquina virtual y una vez generada, sus valores no se modifican.

Usuario y BuzonEntrada: Los usuarios hacen sus solicitudes en un buzón compartido, por lo que para evitar que varios hilos modifiquen su contenido al mismo tiempo, se utiliza synchronized. Cuando el buzón está lleno, los usuarios esperan mediante wait() hasta que se libere espacio y puedan continuar poniendo las solicitudes.

BuzonEntrada y Solicitud: El buzón guarda las solicitudes en un ArrayList y las retira siguiendo el orden FIFO. En este caso, las acciones de agregar y retirar están sincronizadas para evitar que varios hilos modifiquen la lista simultáneamente

BuzonEntrada y GestorSolicitudes: Los gestores retiran las solicitudes mediante el método retirar(), que está sincronizado para evitar que una misma solicitud sea procesada por varios gestores. Cuando el buzón está vacío, los gestores realizan espera semi-activa utilizando Thread.yield(), hasta que aparezcan nuevas solicitudes o termine la producción.

GestorSolicitudes y ZonaEspera: Antes de procesar una solicitud, los gestores deben obtener permiso de la zona de espera; si se alcanza el umbral establecido, quedan en espera mediante wait() hasta que se resuelvan las máquinas virtuales pendientes, lo cual evita que se sigan creando máquinas mientras ya hay demasiadas esperando los recursos

GestorSolicitudes y MaquinaVirtual: Los gestores crean e inician las máquinas virtuales utilizando start(). Cada máquina funciona como un hilo independiente, por lo que no es necesario que el gestor espere a que termine su ejecución para continuar procesando otras solicitudes.

MaquinaVirtual y Solicitud: Cada máquina virtual consulta los recursos que necesita a través de los métodos de su solicitud y como estos datos no se modifican durante el procesamiento, no es necesario implementar sincronización adicional para poder consultarlos

MaquinaVirtual y ZonaEspera: EN el caso de que una máquina no consiga los recursos necesarios, se registra en la zona de espera y se usa wait() para esperar pasivamente, al momento en el que cambia la disponibilidad de recursos, se utiliza notifyAll() para despertar a las máquinas pendientes. Además, se mantiene un contador de cambios para evitar que se pierdan notificaciones.

MaquinaVirtual y Controladores: Las máquinas virtuales solicitan y liberan recursos mediante los controladores de procesadores, memoria y almacenamiento, por lo que cada controlador utiliza métodos synchronized para evitar que varias máquinas reciban los mismos recursos; si una máquina no consigue completar su reserva, libera los recursos que ya había obtenido.

Implementación de la espera total - Gestores

En nuestra implementación, la espera total se controla desde la clase ZonaEspera, antes de retirar una solicitud del buzón, cada gestor utiliza el método esperarPermisoProcesamiento() para comprobar si puede continuar, para esto, se tiene en cuenta tanto la cantidad de máquinas virtuales registradas en espera como las que todavía están en proceso de creación.

Cuando se alcanza el umbral, los gestores paraan de procesar nuevas solicitudes y esperan mediante wait() hasta que las máquinas pendientes puedan conseguir los recursos que necesitan. Mientras esto ocurre, se va actualizando la zona de espera y se utiliza notifyAll() para avisar a los threads que están esperando. Al momento de que ya no quedan máquinas pendientes ni en proceso de creación, los gestores pueden volver a procesar solicitudes del buzón, de esta manera se evita que se acumulen demasiadas máquinas virtuales y permite que el sistema gestione de mejor forma los recursos disponibles.

Validación de la implementación

Para poder comprobar el funcionamiento de nuestro programa, se realizó una ejecución con 3 usuarios, 2 gestores, un buzón de capacidad 3 y una zona de espera con capacidad 3 y umbral

2. Además, el sistema contaba con 4 procesadores, 8 GiB de memoria y 8 GiB de almacenamiento usando el archivo config.txt.

Generación de solicitudes: Se verificó que los usuarios generaran las solicitudes correspondientes a sus identificadores. En total, se esperaban 6 solicitudes y se crearon las 6 máquinas virtuales correspondientes.

Funcionamiento del buzón: Durante la ejecución se pudieron observar situaciones en las que el buzón alcanzó su capacidad máxima. En estos casos, los usuarios tuvieron que esperar hasta que los gestores retiraran solicitudes y quedara espacio disponible.

Asignación de recursos: Se comprobó que las máquinas virtuales intentaran obtener procesadores, memoria y almacenamiento antes de comenzar su ejecución, hubo algunas que no consiguieron los recursos inicialmente y tuvieron que esperar hasta que otras máquinas los liberaran

Zona de espera y sincronización: Se observaron máquinas virtuales que entraron en espera pasiva y posteriormente despertaron para volver a intentar reservar recursos, también se registró la activación del umbral, que restringió temporalmente la aceptación de nuevas solicitudes por parte de los gestores

Terminación del programa: Al finalizar la ejecución, los usuarios, gestores y máquinas virtuales habían terminado, por lo que el buzón y la zona de espera quedaron vacíos y los controladores registraron cero recursos asignados.

En general, la ejecución permitió comprobar que el sistema procesó las 6 solicitudes esperadas y terminó con todos los recursos liberados, cumpliendo las condiciones finales establecidas en el enunciado

Ejecución de Main:

```
==========================================================

 SIMULADOR DE GESTIÓN DE RECURSOS PARA MÁQUINAS VIRTUALES

==========================================================
```

Universidad de los Andes

Configuración:

Usuarios: 3 (solicitudes esperadas: 6)

Gestores: 2

Buzón: capacidad 3

Zona de espera: capacidad 3 | umbral 2

Procesadores: 4

Memoria: 8 GiB | página: 1024 KiB

Almacenamiento: 8 GiB | bloque: 1024 KiB

INICIO DE LA EJECUCIÓN

[Usuario-1] Inicia; generará 1 solicitudes.

[Usuario-3] Inicia; generará 3 solicitudes.

[Usuario-2] Inicia; generará 2 solicitudes.

[Gestor-2] Inicia el procesamiento de solicitudes.

[Gestor-1] Inicia el procesamiento de solicitudes.

[Usuario-3] Generó 3-1 (CPU=3, memoria=1 GiB, almacenamiento=6 GiB).

[Usuario-1] Generó 1-1 (CPU=2, memoria=4 GiB, almacenamiento=6 GiB).

[Usuario-3] Depositó 3-1 en el buzón.

[Usuario-3] Generó 3-2 (CPU=2, memoria=4 GiB, almacenamiento=1 GiB).

[Usuario-3] Depositó 3-2 en el buzón.

Universidad de los Andes

[Usuario-3] Generó 3-3 (CPU=2, memoria=1 GiB, almacenamiento=2 GiB).

[Usuario-3] Depositó 3-3 en el buzón.

[Usuario-2] Generó 2-1 (CPU=1, memoria=2 GiB, almacenamiento=1 GiB).

[Usuario-1] Depositó 1-1 en el buzón.

[Zona de espera] Se alcanzó el umbral. No se admiten más solicitudes.

[Buzón] Está lleno; Usuario-2 espera para depositar.

[Usuario-1] Terminó de producir solicitudes.

[Gestor-1] Retiró la solicitud 1-1 del buzón.

[Usuario-3] Terminó de producir solicitudes.

[Usuario-2] Depositó 2-1 en el buzón.

[Usuario-2] Generó 2-2 (CPU=1, memoria=4 GiB, almacenamiento=2 GiB).

[Buzón] Está lleno; Usuario-2 espera para depositar.

[Gestor-2] Retiró la solicitud 3-1 del buzón.

[Gestor-2] Crea e inicia la MV MV-3-1.

[Gestor-1] Crea e inicia la MV MV-1-1.

[MV-3-1] Inicia. Necesita CPU=3, memoria=1 GiB, almacenamiento=6 GiB.

[MV-1-1] Inicia. Necesita CPU=2, memoria=4 GiB, almacenamiento=6 GiB.

[MV-1-1] No obtuvo procesadores.

[MV-1-1] Espera pasivamente un cambio en la disponibilidad.

[MV-3-1] Reservó todos los recursos y puede comenzar.

[MV-3-1] Ejecutándose durante 11163 ms.

[MV-1-1] Despertó; volverá a intentar desde los procesadores.

[MV-3-1] Liberó todos sus recursos.

[MV-3-1] Finalizó.

[Zona de espera] Todas las MVs pendientes se resolvieron. Se habilita un nuevo grupo.

[MV-1-1] Reservó todos los recursos y puede comenzar.

[MV-1-1] Ejecutándose durante 8744 ms.

[Gestor-2] Retiró la solicitud 3-2 del buzón.

[Gestor-2] Crea e inicia la MV MV-3-2.

[Zona de espera] Se alcanzó el umbral. No se admiten más solicitudes.

[Gestor-1] Retiró la solicitud 3-3 del buzón.

[Usuario-2] Depositó 2-2 en el buzón.

[Gestor-1] Crea e inicia la MV MV-3-3.

[MV-3-2] Inicia. Necesita CPU=2, memoria=4 GiB, almacenamiento=1 GiB.

[Usuario-2] Terminó de producir solicitudes.

[MV-3-3] Inicia. Necesita CPU=2, memoria=1 GiB, almacenamiento=2 GiB.

[MV-3-3] No obtuvo procesadores.

[MV-3-3] Espera pasivamente un cambio en la disponibilidad.

[MV-3-2] Reservó todos los recursos y puede comenzar.

[MV-3-2] Ejecutándose durante 13780 ms.

[MV-1-1] Liberó todos sus recursos.

[MV-1-1] Finalizó.

Universidad de los Andes

[MV-3-3] Despertó; volverá a intentar desde los procesadores.

[MV-3-3] Reservó todos los recursos y puede comenzar.

[Zona de espera] Todas las MVs pendientes se resolvieron. Se habilita un nuevo grupo.

[Gestor-2] Retiró la solicitud 2-1 del buzón.

[Zona de espera] Se alcanzó el umbral. No se admiten más solicitudes.

[Gestor-1] Retiró la solicitud 2-2 del buzón.

[Gestor-1] Crea e inicia la MV MV-2-2.

[MV-3-3] Ejecutándose durante 27288 ms.

[Gestor-2] Crea e inicia la MV MV-2-1.

[Gestor-2] Termina: todos los usuarios finalizaron y el buzón está vacío.

[Gestor-1] Termina: todos los usuarios finalizaron y el buzón está vacío.

[MV-2-2] Inicia. Necesita CPU=1, memoria=4 GiB, almacenamiento=2 GiB.

[MV-2-2] No obtuvo procesadores.

[MV-2-2] Espera pasivamente un cambio en la disponibilidad.

[MV-2-1] Inicia. Necesita CPU=1, memoria=2 GiB, almacenamiento=1 GiB.

[MV-2-1] No obtuvo procesadores.

[MV-2-1] Espera pasivamente un cambio en la disponibilidad.

[MV-2-2] Despertó; volverá a intentar desde los procesadores.

[MV-2-1] Despertó; volverá a intentar desde los procesadores.

[MV-3-2] Liberó todos sus recursos.

[MV-3-2] Finalizó.

[MV-2-2] Reservó todos los recursos y puede comenzar.

[MV-2-2] Ejecutándose durante 38839 ms.

[MV-2-1] Reservó todos los recursos y puede comenzar.

[MV-2-1] Ejecutándose durante 14404 ms.

[MV-2-1] Liberó todos sus recursos.

[MV-2-1] Finalizó.

[MV-3-3] Liberó todos sus recursos.

[MV-3-3] Finalizó.

[MV-2-2] Liberó todos sus recursos.

[MV-2-2] Finalizó.

==========

RESUMEN Y VALIDACIÓN DEL ESTADO FINAL

==========

Solicitudes esperadas: 6

Máquinas virtuales creadas: 6

Usuarios finalizados: true

Gestores finalizados: true

Todas las MVs finalizaron: true

Buzón vacío y sin usuarios activos: true

Zona de espera vacía: true (pendientes registrados: 0)

Universidad de los Andes

Procesadores completamente libres: true (asignados: 0)

Memoria completamente libre: true (páginas asignadas: 0)

Almacenamiento completamente libre: true (bloques asignados: 0)

RESULTADO: todas las condiciones finales se cumplieron.

==========